הנדסאים באריאל

פרויקט גמר

מערכת Help Desk

Full Stack TypeScript – ניהול קריאות שירות

המחלקה להנדסת תוכנה

ברק שרעבי

baraks@ariel.ac.il

1. נקודת התחלה – חובה לקרוא

⚠ בפרויקט זה אין להתחיל פרויקט חדש מאפס.

אתם מקבלים כבסיס את הפרויקט האחרון שבוצע בקורס – מערכת ניהול המשתמשים. מטרת פרויקט הגמר היא לקחת את המבנה, העקרונות והקוד שכבר בניתם ולפתח מהם מערכת חדשה ומורכבת יותר.
מותר ואף רצוי לעשות שימוש חוזר בקוד כללי שכבר קיים בפרויקט הבסיס, כל עוד אתם מבינים אותו ומתאימים אותו למערכת החדשה. בין היתר, ניתן להמשיך להשתמש בעקרונות ובחלקים שכבר קיימים בפרויקט:

- מבנה Vite + React + TypeScript.
- TailwindCSS.
- Radix UI.
- Loading.
- ErrorMessage.
- מבנה של Components עם Props מטופסים.
- State מטופס.
- useEffect לטעינת מידע.
- מבנה services בצד הלקוח.
- פונקציית הבקשות הכללית הקיימת בפרויקט המבוססת על fetch.
- ApiResponse<T>.
- טיפול בתשובת Success ובתשובת Error.
- VITE_API_URL.
- Express + TypeScript.
- CORS.
- מבנה server / db / type.
- MongoDB Native Driver.
- Collection מטופס.
- פעולות Database נפרדות.
- המרה בין MongoDB Document לבין מידע המוחזר ללקוח.
- בדיקת ObjectId.
- קבלת מידע חיצוני כ-unknown.
- Type Guards ופונקציות Parsing/Validation בצד השרת.
- Dialog ו-AlertDialog של Radix UI.

אין חובה לשמור את שמות הקבצים הקיימים, אך יש לשמור על אותם עקרונות של הפרדת אחריות.
הפרויקט החדש אינו "מערכת משתמשים עם שמות אחרים". עליכם להתאים את מבנה הנתונים, הקומפוננטות, הפעולות וה-UI למערכת Help Desk.

2. מטרת הפרויקט

בפרויקט זה תפתחו מערכת Full Stack מלאה לניהול קריאות שירות מסוג Help Desk.
המערכת תאפשר ללקוח לפתוח קריאת שירות דרך אתר ציבורי, ולמנהל המערכת להתחבר לאזור ניהול שבו ניתן לצפות בקריאות, לחפש ולסנן אותן, לצפות בפרטים, לעדכן את מצב הטיפול, למחוק קריאות ולייצא את הנתונים לקובץ CSV.
המטרה המרכזית אינה רק לגרום למערכת לעבוד.
המטרה היא לבנות מערכת שבה כל שכבה ברורה ומטופסת:

Form → React State → Component → API Service → HTTP Request → Express → Database Action

→ MongoDB → HTTP Response → API Service → React State → UI

בכל שלב צריך להיות ברור:

- איזה מידע נכנס.
- איזה מידע יוצא.
- מה הטיפוס שלו.
- האם המידע יכול להיות חסר.
- האם המידע יכול להיות לא תקין.
- כיצד מטפלים בשגיאה.

3. טכנולוגיות חובה

צד לקוח – חובה להשתמש ב:

- Vite
- React
- TypeScript
- TailwindCSS
- Radix UI
- React Router
- fetch באמצעות שכבת ה-API הקיימת בפרויקט הבסיס

הפרויקט שקיבלתם כבר מכיל שכבת API המבוססת על fetch ופונקציית בקשות כללית. יש להמשיך את אותה גישה ולהרחיב אותה לפי צורכי המערכת החדשה.

צד שרת – חובה להשתמש ב:

- Node.js
- Express
- TypeScript
- MongoDB
- MongoDB Native Driver
- CORS

לצורך מערכת ההתחברות יש להשתמש במנגנון Authentication שנלמד/יינתן בקורס. הסיסמה של המנהל חייבת להישמר בצורה מאובטחת ולא כטקסט רגיל.

ספריות אופציונליות: ניתן להשתמש בספריות נוספות שנלמדו בקורס או אושרו מראש.

אין חובה להשתמש ב:

- Zod
- TanStack Table
- Mongoose
- Redux

4. מבנה המערכת

המערכת מורכבת משני אזורים.

אזור ציבורי – משתמש שאינו מחובר יכול:

1. להגיע לדף פתיחת קריאה.
2. למלא טופס.
3. לקבל Validation.
4. לשלוח קריאה לשרת.
5. להגיע לדף אישור לאחר שמירה מוצלחת.

אין צורך ליצור חשבון ללקוח.

אזור ניהול – מנהל המערכת יכול:

1. להתחבר.
2. לצפות ב-Dashboard.
3. לצפות בכל הקריאות.
4. לחפש קריאות.
5. לסנן קריאות.
6. לצפות בפרטי קריאה.
7. לעדכן קריאה.
8. למחוק קריאה.
9. לייצא את הקריאות ל-CSV.
10. להתנתק.

5. שלב האפיון

לפני תחילת הפיתוח יש לעדכן את README.md ולהוסיף תכנון קצר של המערכת.
יש לציין:

- אילו Pages יהיו במערכת.
- מה התפקיד של כל Page.
- אילו Components מרכזיים מתוכננים.
- אילו טיפוסים מרכזיים נדרשים.
- אילו נתונים נשמרים ב-MongoDB.
- אילו Collections קיימים.
- אילו API Endpoints קיימים.
- אילו Endpoints ציבוריים.
- אילו Endpoints מוגנים.
- כיצד מחולקת האחריות בין React, שכבת ה-API, Express ושכבת ה-Database.

אין צורך לכתוב מסמך ארוך. מטרת הסעיף היא להראות שתכננתם את המערכת לפני הפיתוח.

6. React Router ודפי המערכת

פרויקט הבסיס שלכם עבד בעיקר מתוך App.tsx. בפרויקט הגמר יש להרחיב את המבנה ולעבוד עם Pages ו-React Router.

חובה לכלול לפחות את הדפים הבאים:

דף	תפקיד
פתיחת קריאה	טופס ציבורי ליצירת קריאת שירות
דף אישור	הודעה לאחר יצירת קריאה מוצלחת
התחברות מנהל	Login
Dashboard	תמונת מצב כללית
ניהול קריאות	צפייה וניהול של כל הקריאות

ניתן ליצור Pages נוספים אם הם משפרים את מבנה המערכת.

אין לבנות את כל המערכת כ-Component אחד שמציג ומסתיר אזורים באופן ידני.

7. קריאת שירות – המידע הנדרש

כל קריאת שירות תכיל לפחות את המידע הבא:

שדה	משמעות
מזהה	מזהה ייחודי של MongoDB
שם הלקוח	שם מלא
Email	כתובת אימייל
טלפון	מספר טלפון
כותרת	תיאור קצר של הבעיה
תיאור	פירוט התקלה
קטגוריה	סוג התקלה
עדיפות	רמת הדחיפות
סטטוס	מצב הטיפול
תאריך יצירה	מתי נפתחה הקריאה
תאריך עדכון	מתי עודכנה לאחרונה

אין חובה שהטיפוס המייצג קריאה ב-MongoDB יהיה זהה לטיפוס המוחזר ל-Frontend.

8. קטגוריות

הקטגוריה אינה טקסט חופשי.

יש להשתמש בערכים קבועים:

- Hardware
- Software
- Network
- Account
- Other

המערכת אינה אמורה לקבל קטגוריה שאינה אחת מהאפשרויות המוגדרות.

9. Priority

לכל קריאה תהיה רמת עדיפות אחת:

- Low
- Medium
- High

אין לאפשר ערכים אחרים.

10. Status

לכל קריאה יהיה אחד מהמצבים הבאים:

- Open
- In Progress
- Resolved

קריאה חדשה שנפתחת מהטופס הציבורי נוצרת תמיד ב-Open.

הלקוח אינו בוחר Status.

רק אזור הניהול מאפשר לשנות אותו.

דוגמה לזרימת טיפול: Open → In Progress → Resolved

אין חובה למנוע מעבר בחזרה לסטטוס קודם, אלא אם בחרתם להוסיף כלל כזה בעצמכם.

11. דוגמה לקריאה

הדוגמה הבאה מתארת כיצד מידע יכול להיראות בממשק. זו אינה דוגמת קוד ואינה מכתיבה את דרך המימוש.

שדה	ערך
Title	Printer does not print
Customer	Daniel Cohen
Email	daniel@example.com
Phone	050-1234567
Category	Hardware
Priority	High
Status	Open
Description	The printer is connected but does not print documents.
Created	10/03/2026 10: 15
Updated	10/03/2026 10: 15

12. טופס פתיחת קריאה

הדף הציבורי יכלול טופס עם :

- שם מלא.
- Email.
- טלפון.
- כותרת.
- קטגוריה.
- Priority.
- Description.
- כפתור שליחה.

הטופס חייב להיות Controlled Form של React.

ה-State של הטופס חייב להיות מטופס.

אירועי Input ו-Submit חייבים להיות מטופסים.

13. Validation בצד הלקוח

אין לשלוח טופס לא תקין.

חובה לבדוק לפחות:

שדה	דרישות
שם מלא	חובה. אין לקבל ערך שמכיל רק רווחים.
Email	חובה. חייב להיות במבנה סביר של Email.
טלפון	חובה. חייב להכיל ערך סביר עבור מספר טלפון.
כותרת	חובה. לא יכולה להיות ריקה או להכיל רק רווחים. יש לדרוש כותרת בעלת משמעות ולא תו יחיד.
קטגוריה	חובה. חייבת להיות אחת הקטגוריות המוגדרות.
Priority	חובה. חייב להיות אחד מערכי העדיפות המוגדרים.
Description	חובה. צריך להכיל תיאור משמעותי של הבעיה.

14. הודעות שגיאה בטופס

כאשר שדה אינו תקין, יש להציג למשתמש הודעה ברורה.
לדוגמה: "Please enter a valid email address." או: "Description is required."
אין להסתפק בכך שהטופס אינו נשלח.
המשתמש צריך להבין מה עליו לתקן.

15. Validation בצד השרת

בפרויקט הבסיס כבר עבדתם עם מידע שמגיע לשרת כמידע חיצוני, בדיקות על unknown ופונקציות Parsing.
יש להמשיך באותה גישה. גם אם React ביצע Validation, השרת חייב לבדוק את המידע לפני שמירתו.

אין לשמור:

- שדות חובה חסרים.
- שדות שמכילים רק רווחים.
- Category לא חוקית.
- Priority לא חוקי.
- Status לא חוקי.
- מידע שאינו מתאים למבנה שהשרת מצפה לקבל.

⚠ TypeScript אינו מהליך Runtime Validation.

16. יצירת קריאה

לאחר Submit תקין:

1. ה-Frontend שולח את המידע דרך שכבת ה-API.
2. השרת מבצע Validation.
3. הקריאה נשמרת ב-MongoDB.
4. השרת קובע את ה-Status ההתחלתי.
5. השרת קובע את תאריך ושעת היצירה.
6. השרת קובע את תאריך ושעת העדכון הראשוני.
7. מוחזרת תשובה מתאימה.
8. רק לאחר הצלחה המשתמש עובר לדף האישור.

במקרה של כישלון אין לעבור לדף האישור.

17. דף האישור

לאחר יצירה מוצלחת יש להעביר את המשתמש אוטומטית לדף אישור.

הדף יציג הודעה ברורה, למשל: "Your support ticket was submitted successfully".

ניתן להציג גם את מזהה הקריאה שנוצר.

18. MongoDB

יש להשתמש ב-MongoDB Native Driver, כפי שנעשה בפרויקט הבסיס.

חובה לעבוד לפחות עם:

- Collection לקריאות.
- Collection למנהלים.

שמות מומלצים: tickets, admins.

19. Typed Collections

כמו בפרויקט הבסיס, גם בפרויקט הגמר אין להשתמש ב-Collection שאינו מטופס.

לכל Collection צריך להיות ברור איזה Document הוא מכיל.

יש להבדיל, לפי הצורך, בין:

- מידע שה-Frontend שולח.
- Document שנשמר ב-MongoDB.
- מידע ציבורי שמוחזר ל-Frontend.
- מידע לעדכון.
- מידע להתחברות.

אין דרישה שכל המידע הזה יהיה מיוצג באמצעות אותו Interface.

20. שכבת Database

יש לשמור על העיקרון שכבר קיים בפרויקט: Express → Database Actions → MongoDB
פעולות MongoDB אינן אמורות להיות מפוזרות בתוך כל Route.
יש ליצור פעולות Database מסודרות עבור הפעולות הנדרשות במערכת.
כל פונקציה אסינכרונית צריכה להגדיר טיפוס חזרה מתאים.

21. ObjectId

בפרויקט הבסיס כבר טיפלתם במזהי MongoDB.
גם כאן:

- מזהה שמתקבל ב-URL חייב להיבדק.
- אין להניח שכל String הוא ObjectId תקין.
- מזהה לא תקין לא אמור לגרום לקריסת השרת.

יש לטפל בצורה תקינה גם במקרה שבו המזהה חוקי מבחינת המבנה אך הקריאה אינה קיימת.

22. API Response אחיד

פרויקט הבסיס כולל מבנה גנרי של תשובות API עם Success ו-Error.
יש להמשיך להשתמש באותו עיקרון.
אין צורך להמציא מבנה תשובות חדש בכל Endpoint.
תשובה מוצלחת ותשובת שגיאה צריכות להיות ברורות ועקביות לאורך המערכת.
ה-Frontend צריך להשתמש בטיפוסים של תשובות ה-API ולא ב-any.

23. שכבת API בצד הלקוח

בפרויקט הבסיס קיימת תיקיית services ופונקציה כללית המבצעת בקשות באמצעות fetch.
יש להמשיך להשתמש בגישה זו. אין לבצע fetch ישירות בכל Component.
המבנה הרצוי מבחינת אחריות: Component / Page → API Service → Server
שכבת ה-API תהיה אחראית על פעולות כגון:

- יצירת קריאה.
- קבלת כל הקריאות.
- קבלת קריאה לפי ID.
- עדכון קריאה.
- מחיקת קריאה.
- Login.
- Export לפי הצורך.

הפונקציה הכללית הקיימת לבקשות יכולה לשמש בסיס ולהתרחב בהתאם לצורכי הפרויקט.

24. REST API – דרישות חובה

השרת חייב לספק לפחות את הפעולות הבאות:

Method	Endpoint	פעולה	גישה
POST	/tickets	יצירת קריאה	ציבורית
POST	/admin/login	התחברות מנהל	ציבורית
GET	/tickets	כל הקריאות	מנהל
GET	/tickets/:id	קריאה בודדת	מנהל
PATCH	/tickets/:id	עדכון קריאה	מנהל
DELETE	/tickets/:id	מחיקת קריאה	מנהל
GET	/tickets/export/csv	ייצוא CSV	מנהל

ניתן לבחור Prefix כגון /api, אך אם בחרתם – יש להשתמש בו באופן עקבי.

25. GET – כל הקריאות

מסך הניהול יציג את כל הקריאות שנשמרו.

בעת טעינת המסך:

- יש לבצע בקשה לשרת.
- יש להציג Loading בזמן ההמתנה.
- במקרה הצלחה יש לעדכן State מטופס.
- במקרה כישלון יש להציג Error.
- אם אין קריאות יש להציג Empty State.

אין לבצע Refresh ידני לצורך הצגת הנתונים.

26. GET – קריאה בודדת

כמו UserDetailsDialog בפרויקט הבסיס, גם בפרויקט זה נדרש לאפשר צפייה בפרטים מלאים של קריאה.
כאשר המשתמש בוחר לצפות בקריאה, יש לקבל את הקריאה לפי ה-ID שלה מהשרת.
אין להסתפק רק בפרטים שכבר נמצאים בכרטיס או בשורה ברשימה.
יש לטפל ב:

- Loading.
- Error.
- קריאה שאינה קיימת.
- הצגת פרטים לאחר הצלחה.

ניתן להשתמש ב-Radix Dialog לצורך ההצגה.

27. PATCH – עדכון קריאה

זהו אחד החלקים החדשים המרכזיים בפרויקט.
מנהל יכול לעדכן לפחות:

- Status.
- Priority.

לאחר עדכון מוצלח:

- MongoDB חייב להכיל את הערכים החדשים.
- updatedAt חייב להשתנות.
- ה-Frontend חייב להציג את המידע החדש.
- אין לבצע Refresh ידני.

יש להגדיר טיפוס נפרד שמתאר את המידע שמותר לעדכן.
אין לאפשר ל-Frontend לשנות באופן חופשי כל שדה במסמך.

28. DELETE – מחיקת קריאה

המנהל יכול למחוק קריאה. לפני המחיקה חובה לבקש אישור.
ניתן ומומלץ להתבסס על DeleteUserDialog שכבר קיים בפרויקט הבסיס ולהתאים אותו לקריאות.
ה-Dialog יכלול לפחות:

- הסבר מה עומד להימחק.
- Cancel.
- Delete.
- מצב Loading בזמן המחיקה.
- Error במקרה שהמחיקה נכשלת.

לאחר הצלחה הקריאה תיעלם מה-State ומה-UI ללא Refresh.

29. Radix UI

בפרויקט הבסיס כבר השתמשתם ב-Radix UI.
יש להמשיך להשתמש בו כחלק פונקציונלי מהמערכת.
חובה להשתמש לפחות בשני Primitives שונים.
שימושים מתאימים:

- Dialog – פרטי קריאה.
- AlertDialog – אישור מחיקה.
- Select – Category, Priority או Status.
- Popover – אם הוא מתאים לפיצ'ר שבחרתם.

אין חובה להשתמש בכל האפשרויות.
השימוש צריך להיות אמיתי ולא רכיב שהוכנס רק כדי לעמוד בדרישה.

30. Loading ו-Error Components

בפרויקט הבסיס קיימים Components כלליים ל-Loading ול-Error.
אין צורך לכתוב אותם מחדש אם הם מתאימים.
מותר להשתמש בהם ולהרחיב אותם.
יש להציג Loading/Error לפחות ב:

- טעינת רשימת הקריאות.
- טעינת קריאה בודדת.
- יצירת קריאה.
- עדכון.
- מחיקה.
- Login.

אין להסתפק ב-console.log.

31. Empty State

כאשר אין קריאות: "No support tickets were found."
כאשר חיפוש או Filter אינם מחזירים תוצאות: "No tickets match the current search and filters."
אין להציג אזור ריק ללא הסבר.

Dashboard .32

לאחר התחברות מנהל, יש להציג Dashboard.

הוא יציג לפחות:

- Total Tickets.
- Open.
- In Progress.
- Resolved.
- High Priority.

לדוגמה:

מדד	ערך
Total Tickets	42
Open	13
In Progress	18
Resolved	11
High Priority	7

אין חובה ליצור Endpoint חדש עבור הנתונים האלה.

אם כל המידע כבר קיים בצד הלקוח, ניתן לחשב את המדדים מתוך מערך הקריאות.

המטרה היא להשתמש בידע הקיים שלכם על Arrays, filter, State ו-Rendering.

33. מסך ניהול הקריאות

המסך יציג את הקריאות בצורה מסודרת.

ניתן לבחור:

- Cards.
- Table.
- שילוב בין שניהם.

אין חובה להשתמש בספריית Table.

לכל קריאה יש להציג לפחות מידע המאפשר לזהות אותה:

- Title.
- Customer.
- Category.
- Priority.
- Status.
- Created Date.

הפרטים המלאים יכולים להופיע ב-Dialog או Page מתאים.

34. Search

יש להוסיף Search בצד הלקוח.

החיפוש צריך לעבוד לפחות לפי:

- Title.
- Customer Name.
- Email.

אין חובה ליצור Endpoint חיפוש בצד השרת.

החיפוש יכול להתבצע על מערך הקריאות שכבר נטען.

35. Filters

יש לאפשר Filter לפחות לפי:

Filter	ערכים
Status	All Open In Progress Resolved
Priority	All Low Medium High
Category	All Hardware Software Network Account Other

יש לאפשר שילוב בין Search לבין Filters.

דוגמה: Search: printer · Status: Open · Priority: High · Category: Hardware

יש להציג רק קריאות המתאימות לכל התנאים שנבחרו.

36. Authentication – Admin

המערכת תכלול Admin אחד לפחות.

אין צורך לבנות Registration למנהל.

המנהל קיים מראש במערכת.

יש לשמור לפחות:

- Username.
- Password בצורה מאובטחת.

אין לשמור Password כטקסט רגיל.

37. Login

דף Login יכלול:

- Username.
- Password.
- כפתור Login.
- Loading.
- Error.

Login שגוי יציג הודעה כללית, לדוגמה: "Invalid username or password."

אין להציג למשתמש איזה מבין שני השדות היה שגוי.

38. אזור ניהול מוגן

הדרישות של המטלה המקורית כוללות ממשק Admin שאליו מגיעים רק לאחר Login.
לכן:

- משתמש שאינו מחובר אינו יכול לגשת למידע הניהולי.
- הסתרת כפתור ב-React אינה מספיקה.
- גם השרת חייב להגן על ה-Endpoints הניהוליים.
- בקשה לא מורשית חייבת להידחות.
- לאחר Login מוצלח, הבקשות המוגנות צריכות להכיל את פרטי ההרשאה הנדרשים.
- לאחר Logout יש להסיר את מצב ההתחברות בצד הלקוח.

מנגנון ההתחברות צריך לפעול לפי הדרך שנלמדה בקורס.

39. Protected Pages

העמודים הבאים הם עמודי Admin:

- Dashboard.
- Tickets Management.

משתמש שאינו מחובר שמנסה להגיע אליהם ישירות צריך לעבור למסך Login.
React Router אחראי על הניווט בצד הלקוח.
השרת עדיין אחראי על אבטחת המידע עצמו.

40. Logout

יש להוסיף כפתור Logout.
לאחר Logout:

- מצב ההתחברות המקומי מוסר.
- המשתמש מופנה לעמוד Login או לעמוד הציבורי.
- בקשות ניהול נוספות אינן אמורות להצליח ללא Login מחדש.

41. CSV Export

הדרישה לייצוא נתונים מהמטלה הקודמת נשמרת.

במסך הניהול יהיה כפתור : "Export CSV"

לחיצה על הכפתור תוריד קובץ CSV אמיתי המכיל את הקריאות.

הקובץ יכלול לפחות :

- Ticket ID
- Title
- Customer Name
- Email
- Phone
- Category
- Priority
- Status
- Description
- Created At
- Updated At

יש לכלול שורת כותרות וסדר עמודות ברור ועקבי.

פעולת הייצוא היא פעולה מוגנת למנהל בלבד.

42. תאריך ושעה

השרת אחראי על :

- createdAt
- updatedAt

אין לקבל את התאריכים מהלקוח בעת יצירת קריאה.

בעת יצירה שני הערכים מתארים את זמן היצירה.

בעת Update יש לעדכן את זמן העדכון.

ב-Frontend יש להציג את התאריך והשעה בצורה קריאה למשתמש.

43. HTTP Status Codes

יש להשתמש ב-Status Codes בהתאם לתוצאה.

יש להכיר ולהשתמש לפי הצורך לפחות ב:

- OK 200
- Created 201
- No Content 204
- Bad Request 400
- Unauthorized 401
- Forbidden 403
- Not Found 404
- Internal Server Error 500

אין להחזיר 200 עבור כל מצב.

דוגמאות:

- יצירת קריאה מוצלחת אינה זהה לקריאה שאינה קיימת.
- Login שגוי אינו שגיאת Server.
- ID לא תקין אינו הצלחה.
- Error של MongoDB אינו Validation Error.

44. TypeScript – דרישת חובה מרכזית

⚠ אין להשתמש ב-any ואין להשתמש ב-as any כדי לעקוף את TypeScript.

הפרויקט חייב לעבוד עם strict: true.

הקוד צריך לעבור Build/Type Checking ללא שגיאות.

45. טיפוסים שנדרשים במערכת

עליכם ליצור את הטיפוסים שהמערכת באמת צריכה.
בין היתר, המערכת תצטרך לתאר:

- Ticket שמוצג ב-Frontend.
- מידע ליצירת Ticket.
- Ticket Document ב-MongoDB.
- מידע שמותר לעדכן.
- Status.
- Priority.
- Category.
- Admin Login.
- API Responses.
- Component Props.
- Callback Props.
- Form State.
- Error/Loading States לפי הצורך.
- Request Params בצד השרת.
- Request Body בצד השרת.

אין חובה להשתמש בדיוק בשמות מסוימים.
הבחירה בשמות ובחלוקה היא חלק מהמטלה.

46. Props

כמו בפרויקט הבסיס, כל Component שמקבל Props חייב לקבל טיפוס מתאים.
גם Callback הוא Prop ולכן גם הוא צריך טיפוס.
לדוגמה רעיונית: TicketCard מקבל Ticket ומקבל פעולות שההורה מאפשר לו לבצע.
אין להעביר Props לא מטופסים.

State .47

State משמעותי חייב להיות מטופס.

יש להתייחס בין היתר ל:

- מערך קריאות.
- קריאה שנבחרה.
- Form.
- Search.
- Filters.
- Loading.
- Error.
- Authentication.

אם State יכול להיות "אין ערך כרגע", הדבר צריך לבוא לידי ביטוי בטיפוס.

React Events .48

אירועי React חייבים להיות מטופסים.

הדבר כולל לפחות:

- Input Change.
- Form Submit.
- Select/Filter changes לפי דרך המימוש.

אין להשתמש ב-any עבור Event.

useEffect .49

כמו בפרויקט הבסיס, יש להשתמש ב-useEffect במקום המתאים לטעינה ראשונית של מידע.

אין לבצע לולאות בקשות בגלל Dependency Array שגוי.

אין לבצע בקשה לשרת בכל Render ללא סיבה.

50. עדכון State ללא Refresh

אחת הדרישות החשובות בפרויקט:

לאחר:

- Create.
- Update.
- Delete.

הממשק צריך להציג את המצב החדש ללא Refresh ידני של הדפדפן.

הדרך שבה תעדכנו את ה-State היא חלק מהפתרון שלכם.

51. TailwindCSS

יש להשתמש ב-TailwindCSS שכבר מותקן בפרויקט.

יש לעצב לפחות:

- Navigation.
- Public Form.
- Login.
- Dashboard.
- Ticket List.
- Ticket Card / Table.
- Buttons.
- Search.
- Filters.
- Dialogs.
- Loading.
- Error.
- Empty State.

אין צורך להוסיף Framework CSS נוסף.

52. Responsive Design

המערכת צריכה להיות שימושית גם במסכים צרים.
יש לוודא לפחות:

- Form אינו יוצא מהמסך.
- Dashboard מסתדר במספר רוחבים.
- Ticket List קריאה.
- Navigation שימושי.
- Buttons אינם נחתכים.
- Dialogs מתאימים למסך.

אין דרישה לבנות אפליקציית Mobile נפרדת.

53. Environment Variables

בפרויקט הבסיס כבר קיים VITE_API_URL. יש להמשיך להשתמש בו.
אין לפזר כתובת Server קשיחה בכל הקבצים.
גם בצד השרת אין לשמור בקוד:

- MongoDB URI אמיתי.
- Password.
- Secret.
- Credentials אחרים.

יש להשתמש ב-Environment Variables.

54. gitignore

אין להעלות ל-GitHub:

- env.
- node_modules
- MongoDB credentials
- Secrets
- Passwords

יש להוסיף .env.example שמציג אילו משתנים נדרשים להרצה, ללא הערכים הסודיים.

55. CORS

בפרויקט הבסיס כבר קיים CORS.
יש להשאיר הגדרת CORS מתאימה כך שה-Frontend יוכל לבצע בקשות ל-Backend.
אין לפתור בעיות CORS על ידי שינוי הגדרות אבטחה בדפדפן.

56. מבנה ה-Frontend

פרויקט הבסיס כולל כבר הפרדה בין components, services, types.
יש לשמור על החלוקה ולהוסיף pages עבור React Router.
מבנה אפשרי מבחינת אחריות – src/:

- /components
- /pages
- /services
- /types
- App.tsx
- main.tsx

אין חובה לשמור שמות קבצים מדויקים.

57. שימוש חוזר ב-Components מהפרויקט הקודם

מותר ואף מומלץ להתבסס על Components שכבר בניתם.
דוגמאות להתאמה:

בפרויקט הבסיס	בפרויקט הגמר
UserForm	Form לפתיחת קריאה
UserList	רשימת קריאות
UserCard	כרטיס/שורת קריאה
UserDetailsDialog	פרטי קריאה
DeleteUserDialog	אישור מחיקת קריאה
Loading	שימוש חוזר
ErrorMessage	שימוש חוזר

הטבלה אינה אומרת שעליכם רק לשנות שמות.
יש להתאים את הלוגיקה, הטיפוסים, השדות וההתנהגות לדרישות החדשות.

58. שימוש חוזר ב-Service

הפרויקט הקודם כבר כולל Service שבו:
- כתובת ה-API מגיעה מ-Environment Variable.
- קיימת פונקציה כללית לביצוע בקשות.
- תשובות השרת מטופסות באמצעות Generic.
- Error Response הופך לשגיאה שה-Component יכול לטפל בה.

יש לשמור על העקרונות האלה.
אין לחזור למצב שבו בכל Component יש קוד Fetch שונה.

59. מבנה ה-Backend

יש להתבסס על המבנה שכבר קיים: /server, /db, /type.
יש לשמור על הפרדת אחריות.
מבנה אפשרי – /Server:
- /server
- /db
- /type
- package.json
- tsconfig.json

ניתן להוסיף תיקיות נוספות, למשל לצורך Authentication, אם הן משפרות את המבנה.
אין חובה לעבור ל-MVC או ל-Mongoose.

60. שימוש חוזר בקוד Server

ניתן להתבסס על הדרך שכבר קיימת בפרויקט:
- Express App.
- express.json().
- CORS.
- connectDB().
- typed Request Params.
- typed Response.
- ApiResponse<T>.
- פונקציות Parsing.
- Type Guards.
- Database Actions.
- ObjectId validation.
- פונקציית המרה מ-Database Document למידע שמוחזר ללקוח.

עליכם להתאים את העקרונות ל-Tickets ולא להעתיק User Code ללא שינוי מהותי.

61. טיפול בשגיאות

יש לטפל לפחות במצבים הבאים:

- שגיאה בחיבור ל-MongoDB.
- יצירת קריאה עם מידע לא תקין.
- ID לא תקין.
- Ticket שאינו קיים.
- Loading שנכשל.
- Update שנכשל.
- Delete שנכשל.
- Login שנכשל.
- בקשה מוגנת ללא הרשאה.
- CSV Export שנכשל.

אין לחשוף למשתמש פרטי מערכת פנימיים שאין לו צורך לדעת.

62. GitHub

כמו במטלת הסיום הקודמת, העבודה חייבת להיות מתועדת במאגר GitHub ציבורי.

יש לעבוד עם Git לאורך הפרויקט.

אין להעלות את כל הפרויקט ב-Commit יחיד ביום ההגשה.

יש לבצע Commit לכל שינוי משמעותי.

דוגמאות לשינויים משמעותיים:

- מעבר ממערכת Users ל-Tickets.
- הוספת Ticket Form.
- הוספת Router.
- הוספת Update.
- הוספת Dashboard.
- הוספת Authentication.
- הוספת CSV.
- השלמת Validation.

README .63

README.md הוא חלק מהציון.

עליו לכלול לפחות:

סעיף	מה לכלול
Project Description	מה המערכת עושה.
Starting Point	ציינו שהפרויקט מבוסס על פרויקט ה-Full Stack האחרון מהקורס ותארו בקצרה אילו חלקים שימשו אתכם כבסיס.
Technologies	רשימת הטכנולוגיות.
Pages	העמודים במערכת.
Project Structure	הסבר קצר על החלוקה לתיקיות.
Data Model	איזה מידע נשמר עבור Ticket ואיזה מידע נשמר עבור Admin.
API	רשימת Endpoints, Method ותפקיד.
Authentication	תיאור כללי של מנגנון ההתחברות והגנת ה-Admin.
Environment Variables	שמות המשתנים הנדרשים, ללא Secrets.
Installation	כיצד מתקינים.
Run	כיצד מריצים Client וכיצד מריצים Server.

64. תרחיש בדיקה מלא

לפני ההגשה ודאו שניתן לבצע את התרחיש הבא מתחילתו ועד סופו:

1. מריצים Client ו-Server.
2. ה-Server מתחבר ל-MongoDB.
3. נכנסים לדף הציבורי.
4. מנסים לשלוח Form ריק.
5. מוצגות הודעות Validation.
6. מזינים Email לא תקין.
7. המערכת מציגה Error מתאים.
8. ממלאים קריאה תקינה.
9. שולחים אותה.
10. בזמן השליחה מוצג מצב Loading.
11. הקריאה נשמרת ב-MongoDB.
12. הקריאה מקבלת Open אוטומטית.
13. התאריך נקבע אוטומטית.
14. עוברים לדף האישור.
15. נכנסים לדף Admin Login.
16. Login שגוי נדחה.
17. Login תקין מצליח.
18. עוברים ל-Dashboard.
19. ה-Dashboard מציג נתונים נכונים.
20. עוברים לרשימת הקריאות.
21. הקריאה החדשה מופיעה.
22. Search מצליח למצוא אותה.
23. Filter לפי Status עובד.
24. Filter לפי Priority עובד.
25. Filter לפי Category עובד.
26. שילוב Search ו-Filters עובד.
27. פותחים את הקריאה לצפייה.
28. הפרטים נטענים לפי ID מהשרת.

29. משנים Status ל-In Progress.
30. הנתון נשמר ב-MongoDB.
31. ה-UI מתעדכן ללא Refresh.
32. משנים Priority.
33. updatedAt משתנה.
34. מסמנים את הקריאה כ-Resolved.
35. מורידים CSV.
36. הקריאה מופיעה בקובץ.
37. לוחצים Delete.
38. מופיע AlertDialog.
39. לוחצים Cancel והקריאה אינה נמחקת.
40. מבצעים Delete שוב ומאשרים.
41. הקריאה נמחקת מה-Database.
42. הקריאה נעלמת מה-UI ללא Refresh.
43. מבצעים Logout.
44. מנסים להגיע ישירות לעמוד Admin.
45. המערכת דורשת Login.
46. ניסיון לבצע בקשה מוגנת ללא הרשאה נדחה גם על ידי השרת.

⚠ אם אחד השלבים אינו עובד, הפרויקט אינו שלם.

65. מקרים מיוחדים שחובה לבדוק

בנוסף לתרחיש הראשי יש לבדוק:

- אין Tickets בכלל.
- Search אינו מוצא תוצאות.
- שילוב Filters אינו מוצא תוצאות.
- MongoDB אינו זמין.
- ID אינו ObjectId תקין.
- ID תקין אך Ticket אינו קיים.
- Request Body אינו תקין.
- Category לא חוקית.
- Priority לא חוקי.
- Status לא חוקי.
- Login שגוי.
- מידע חסר ב-Login.
- בקשה מוגנת ללא Authentication.
- Delete על Ticket שלא קיים.
- Update על Ticket שלא קיים.

66. דברים שאינם נדרשים

כדי לשמור על היקף סביר, אין חובה לבצע:

- Registration ללקוח.
- Registration למנהל.
- Password Reset.
- כמה סוגי משתמשים.
- Roles מורכבים.
- שליחת Email.
- העלאת תמונות.
- העלאת קבצים.
- Chat.
- WebSocket.
- Notifications.
- Pagination.

- Server-side Search.
- Server-side Filtering.
- Redux.
- Context API.
- Mongoose.
- TanStack Table.
- Zod.
- Deployment לענן.

⚠ תוספות אינן מפצות על דרישות חובה שאינן עובדות.

67. כללי קוד

הקוד חייב להיות:

- מסודר.
- מחולק לפי אחריות.
- עם שמות ברורים.
- ללא any.
- ללא as any.
- ללא Secrets.
- ללא שכפול מיותר.
- עם הערות רק כאשר הן באמת מסבירות משהו שאינו ברור מהקוד.

אין צורך לכתוב Comment מעל כל שורה.

68. ניקוד

תחום	ניקוד
TypeScript – טיפוסים, Props, State, API, Server, DB	20
React + Components + useState + useEffect	10
React Router + Pages + Navigation	5
Forms + Client Validation + Server Validation	10
API Service + שימוש נכון ב-fetch וב- ApiResponse<T>	8
Express + REST API + Status Codes	10
MongoDB + Typed Collections + Database Layer	10
Update / PATCH + סנכרון UI	7
Authentication + Admin Area	7
Dashboard + Search + Filters	5
TailwindCSS + Radix UI + Responsive UX	4
CSV Export	2
GitHub + README + Organization	2
סה"כ	100

69. Checklist לפני הגשה

Project Base:

☐ הפרויקט מבוסס על פרויקט ה-Full Stack האחרון.

☐ לא נבנה Vite Project חדש ללא סיבה.

☐ נשמרה שכבת services.

☐ נעשה שימוש בפונקציית Fetch כללית או בהרחבה שלה.

☐ נשמר עיקרון ApiResponse<T>.

☐ נעשה שימוש חוזר ב-Loading/Error במקום לכתוב פתרונות כפולים.

Frontend:

☐ Vite.

☐ React.

☐ TypeScript.

☐ React Router.

☐ TailwindCSS.

☐ Radix UI.

☐ Public Ticket Form.

☐ Controlled Form.

☐ Validation.

☐ Confirmation Page.

☐ Admin Login.

☐ Dashboard.

☐ Ticket Management.

☐ Ticket Details.

☐ Search.

☐ Filters.

☐ Update.

☐ Delete Confirmation.

☐ CSV Export.

☐ Logout.

☐ Loading.

☐ Error.

☐ Empty State.

☐ אין any.

Backend:

☐ Express + TypeScript.

☐ CORS.

☐ MongoDB.

☐ Typed Collections.

☐ Database Actions.

☐ ObjectId Validation.

☐ Create Ticket.

☐ Get Tickets.

☐ Get Ticket By ID.

☐ Update Ticket.

☐ Delete Ticket.

☐ Login.

☐ Protected Admin Endpoints.

☐ CSV Export.

☐ Server Validation.

☐ Status Codes.

☐ unknown מטופל בצורה בטוחה.

☐ אין any.

Database:

☐ Tickets Collection.

☐ Admins Collection.

☐ Created At.

☐ Updated At.

☐ Admin Password אינו נשמר כטקסט רגיל.

Configuration:

☐ VITE_API_URL.

☐ MongoDB URI מגיע מ-Environment Variable.

☐ Secrets אינם בקוד.

☐ env. אינו ב-GitHub.

☐ env.example. קיים.

☐ gitignore. תקין.

GitHub:

☐ Repository ציבורי.

☐ README מלא.

☐ Commit History מסודר.

☐ אין node_modules.

☐ אין Credentials.

☐ ניתן להבין מה-README כיצד להריץ את המערכת.

70. דגש מסכם

פרויקט הגמר הוא המשך ישיר של פרויקט ניהול המשתמשים.

בפרויקט הקודם כבר בניתם:

Typed React Components → Typed State → API Service → fetch → ApiResponse<T> → Express + TypeScript → Validation of external data → Database Actions → Typed MongoDB Collection

בפרויקט הגמר אתם נדרשים לקחת את התשתית הזו ולהרחיב אותה למערכת אמיתית יותר:

Help Desk + Multiple Pages + Search & Filters + Update + Dashboard + Admin Area + CSV

אין צורך להמציא מחדש חלק שכבר פתרנו בקורס כאשר הוא מתאים לשימוש חוזר.

מצד שני, אין להעתיק את פרויקט המשתמשים ולשנות רק שמות.

עליכם להבין את הקוד, להתאים אותו למודל החדש, להוסיף את הדרישות החדשות ולשמור לאורך כל הדרך על TypeScript, הפרדת שכבות וקוד מסודר.

בהצלחה!

הנדסאים
באריאל